

Other Deadlock Issues

1 Main Idea

Scope

Deadlock is not only a problem of ordinary hardware resources. Special cases also appear in databases, communication systems, networks, and scheduling policies.

This note covers four related topics:

- two-phase locking,
- communication deadlocks,
- livelock,
- starvation.

2 Two-Phase Locking

Definition

Two-phase locking is a database locking method where a process first obtains all required locks, then performs its updates and releases the locks.

It is useful when a transaction needs to lock several database records before updating them.

Phase	Action
Phase 1: Growing phase	The process requests locks one by one. No real update work is done.
Phase 2: Shrinking phase	After all locks are acquired, the process updates records and releases locks.

Restart rule

If a needed record is already locked during phase 1, the process releases all locks it already holds and starts phase 1 again.

Two-Phase Locking Pattern

lock everything first → perform updates → release locks

Limitation

Two-phase locking works only if the process can safely stop and restart before doing irreversible work. It is not suitable when the process has already sent network messages, updated files, or performed non-repeatable actions.

3 Communication Deadlocks

Communication Deadlock

A **communication deadlock** occurs when processes wait for messages or replies that only other waiting processes can send.

Example:

- Process A sends a request to process B .
- A blocks waiting for a reply.
- The request message is lost.
- B waits for a request.
- A waits for a reply.

This is a deadlock by the formal definition: each process is waiting for an event only another process can cause. But it is not a classical resource deadlock because no process is holding a resource wanted by the other.

Type	Cause	Typical fix
Resource deadlock	Processes compete for resources such as locks, buffers, or devices.	Ordering, prevention, detection, recovery.
Communication deadlock	Processes wait for missing messages or replies.	Timeouts and retransmission.

4 Timeouts

Timeout idea

When a process sends a message and expects a reply, it starts a timer. If the timer expires before the reply arrives, the process assumes the message was lost and sends it again.

Timeouts can break communication deadlocks because they stop a process from waiting forever.

Duplicate-message danger

If the original message was not lost but only delayed, retransmission may deliver the same request twice. This is dangerous for operations such as banking payments.

Correct communication protocols must handle duplicates, delayed replies, lost messages, and re-transmissions.

5 Resource Deadlock in a Network

Not every network deadlock is a communication deadlock. Routers also contain finite buffers, and buffers are resources.

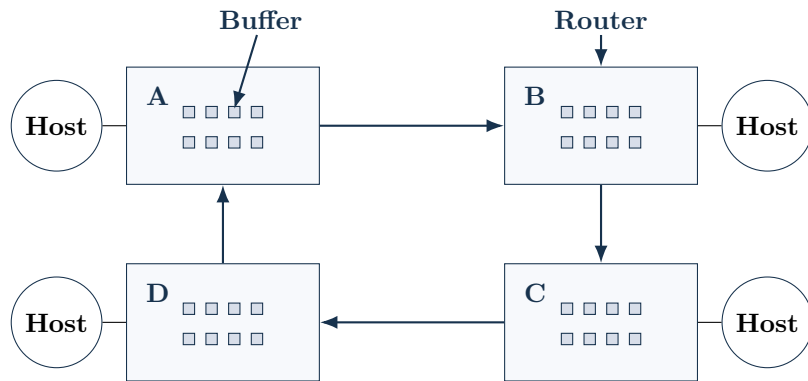


Figure: A resource deadlock in a network. Each router waits for buffer space at the next router.

Network deadlock

If all packets at *A* need to go to *B*, all packets at *B* need to go to *C*, all packets at *C* need to go to *D*, and all packets at *D* need to go to *A*, no packet can move if every next-hop buffer is full.

6 Livelock

Livelock

Livelock occurs when processes keep changing state in response to each other, but no process makes real progress.

In deadlock, processes are blocked. In livelock, processes are active, but their actions do not move the computation forward.

Polite Locking Pattern

```
process A:
  acquire resource_1
  while try_lock(resource_2) fails:
    release resource_1
    wait fixed time
    acquire resource_1

process B:
  acquire resource_2
  while try_lock(resource_1) fails:
    release resource_2
    wait fixed time
    acquire resource_2
```

Why this can livelock

If both processes release and retry at the same time, they may repeatedly collide. Neither is blocked, but neither reaches the section that uses both resources.

7 Another Livelock Example

Suppose a system has 100 process-table slots. Ten programs are running, and each needs to create 12 children.

- Each original process creates 9 children.
- The table now has $10 + 90 = 100$ processes.
- Each original process still needs 3 more children.
- Every `fork` fails because the process table is full.
- Each process waits and retries forever.

Observation

The processes are not blocked on a lock, but they still make no progress. This is livelock.

8 Starvation

Starvation

Starvation occurs when a process waits indefinitely because the resource-allocation policy keeps choosing other processes.

Example: a printer scheduler always chooses the smallest print job first. This maximizes the number of quickly completed jobs, but a huge print job may wait forever if small jobs keep arriving.

Problem	What happens	Typical fix
Starvation	A process is repeatedly postponed even though it is not deadlocked.	First-come, first-served or aging.
Deadlock	Processes wait forever for events only each other can cause.	Prevention, avoidance, detection, recovery.
Livelock	Processes keep running but repeatedly fail to make progress.	Random backoff, priority rules, or asymmetric retry behavior.

9 Starvation vs Deadlock

Key difference

A deadlocked process is blocked and cannot choose another action. A starving process may be repeatedly denied service by policy, but the system as a whole may still be active.

Starvation can be avoided with a first-come, first-served policy: the process waiting the longest receives the resource next. Eventually every request becomes the oldest request.

10 Quick Comparison

Issue	Core cause	Main idea
Two-phase locking	Need to update multiple locked records safely.	Lock first; update only after all locks are held.
Communication deadlock	Lost or missing messages cause processes to wait forever.	Use timeouts and retransmission.
Network resource deadlock	Router buffers are finite resources.	Prevent or recover from buffer cycles.
Livelock	Processes keep retrying politely in sync.	Break symmetry with random delay or ordering.
Starvation	Allocation policy repeatedly favors others.	Use fairness, FCFS, or aging.